

**Course Code: 20CS401**

**Course Title: Database Management Systems**

**Academic Year: 2024 - 2025 (Even Sem)**

## MODULE 3 TRANSACTIONS AND ADVANCED CONCEPTS

**Transaction Concepts - Transaction model - ACID Properties -**

## Transaction Concept

- A transaction is a unit of program execution that accesses and possibly updates various data items
- For example, transaction to transfer \$50 from account A to account B:
- 1. read(A) ✓
- 2.  $A := A - 50$  ✓
- 3. write(A)
- 4. read(B)
- 5.  $B := B + 50$
- 6. write(B)
- Two main issues to deal with:
  - ❖ Failures of various kinds, such as hardware failures and system crashes
  - ❖ Concurrent execution of multiple transactions

→ transaction A → B  
 $A = A - 50$



2)

## Required Properties of a Transaction

### Atomicity requirement

**ACID** property

If the transaction fails after step 3 and before step 6, money will be “lost”

leading to an inconsistent database state

- Failure could be due to software or hardware
- The system should ensure that updates of a partially executed transaction are not reflected in the database

Atomicity  
Consistency

Isolation  
Durability

Transaction to transfer \$50 from account A to account B:

1. ~~read(A)~~
2.  $A := A - 50$
3. ~~write(A)~~
4. ~~read(B)~~
5.  $B := B + 50$
6. ~~write(B)~~

## Required Properties of a Transaction

### Consistency requirement

In example, the sum of A and B is unchanged by the execution of the transaction. In general, consistency requirements include

- Explicitly specified integrity constraints
  - primary keys and foreign keys
- Implicit integrity constraints
  - sum of balances of all accounts, minus sum of loan amounts must equal value of cash-in-hand
- A transaction, when starting to execute, must see a consistent database
- During transaction execution the database may be temporarily inconsistent
- When the transaction completes successfully the database must be consistent
- Erroneous transaction logic can lead to inconsistency
  1. read(A)
  2.  $A := A - 50$
  3. write(A)
  4. read(B)
  5.  $B := B + 50$
  6. write(B)

## Required Properties of a Transaction

### Isolation requirement

If between steps 3 and 6 (of the fund transfer transaction), another transaction T2 is allowed to access the partially updated database, it will see an inconsistent database (the sum  $A + B$  will be less than it should be).

**T1**

1. read(A)
2.  $A := A - 50$
3. write(A)

**T2**

read(A), read(B), print(A+B)

4. read(B)
5.  $B := B + 50$
6. write(B)

- Isolation can be ensured trivially by running transactions **serially**
- That is, one after the other
- However, executing multiple transactions concurrently has significant benefits

## Required Properties of a Transaction

### Durability requirement

Once the user has been notified that the transaction has completed (i.e., the transfer of the \$50 has taken place), the updates to the database by the transaction must persist even if there are software or hardware failures

Transaction to transfer \$50 from account A to account B:

1. read(A)
2.  $A := A - 50$
3. write(A)
4. read(B)
5.  $B := B + 50$
6. write(B)

persistence

## ACID Properties

A transaction is a unit of program execution that accesses and possibly updates various data items. To preserve the integrity of data the database system must ensure:

### Atomicity:

Either all operations of the transaction are properly reflected in the database or none are

### Consistency:

Execution of a transaction in isolation preserves the consistency of the database

### Isolation:

Although multiple transactions may execute concurrently, each transaction must be unaware of other concurrently ~~executing~~ transactions. Intermediate transaction results must be hidden from other concurrently executed transactions. That is, for every pair of transactions  $T_i$  and  $T_j$ , it appears to  $T_i$  that either  $T_j$ , finished execution before  $T_i$  started, or  $T_j$  started execution after  $T_i$  finished

### Durability:

After a transaction completes successfully, the changes it has made to the database persist, even if there are system failures



## ACID Properties

A

Atomic

All changes to the data must be performed successfully or not at all

C

Consistent

Data must be in a consistent state before and after the transaction

I

Isolated

No other process can change the data while the transaction is running

D

Durable

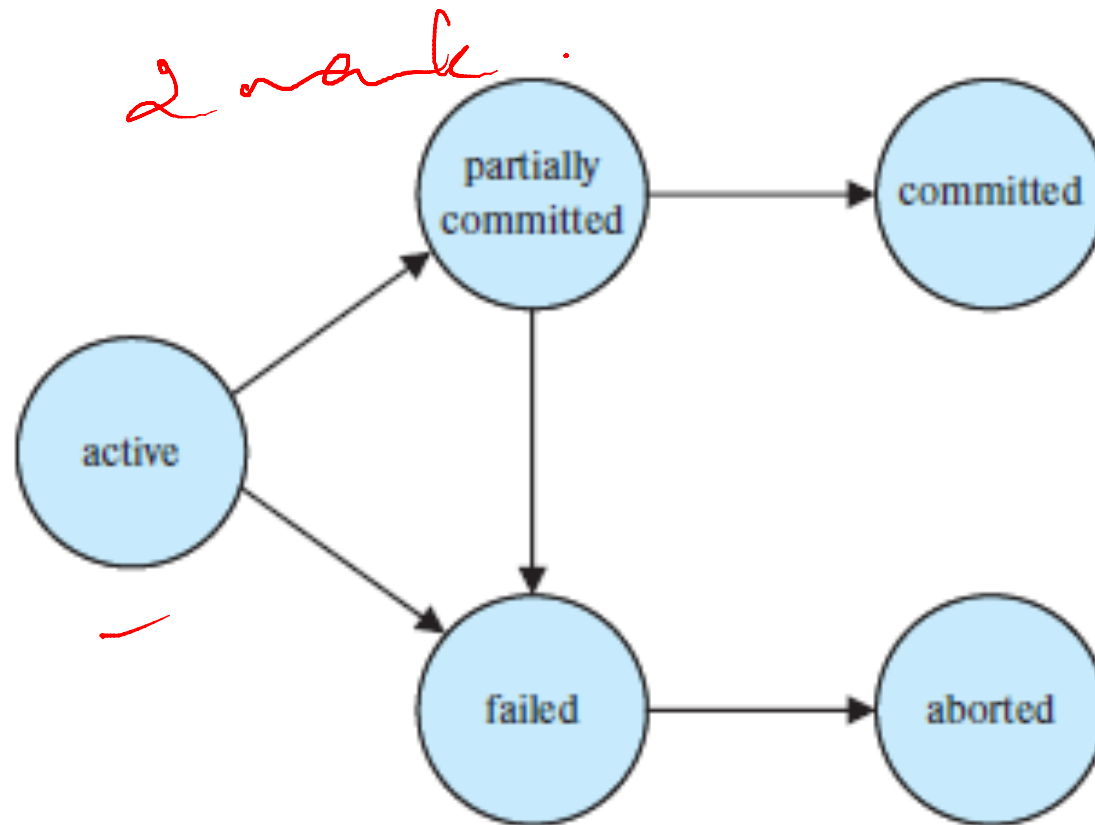
The changes made by a transaction must persist

## TRANSACTION STATE

- **Active**, the initial state; the transaction stays in this state while it is executing
- **Partially committed**, after the final statement has been executed.
- **Failed**, after the discovery that normal execution can no longer proceed.
- **Aborted**, after the transaction has been rolled back and the database restored to its state prior to the start of the transaction.
- Two options after it has been aborted:
  - ☐ restart the transaction – only if no internal logical error
  - ☐ kill the transaction
- **Committed**, after successful completion.

restart  
kill the  
transaction

## TRANSACTION STATES



## **Concurrent Executions**

Multiple transactions are allowed to run concurrently in the system.

Advantages are:

**Increased processor and disk utilization**, leading to better transaction *throughput* E.g. one transaction can be using the CPU while another is reading from or writing to the disk

**Reduced average response time** for transactions: short transactions need not wait behind long ones.

**Concurrency control schemes** – mechanisms to achieve isolation

Prevent them from destroying the consistency of the database

## Concurrent Executions

**Schedule** – a sequences of instructions that specify the chronological order in which instructions of concurrent transactions are executed

A schedule for a set of transactions must consist of all instructions of those transactions

Must preserve the order in which the instructions appear in each individual transaction.

A transaction that successfully completes its execution will have a **commit** instructions as the last statement

By default transaction assumed to execute commit instruction as its last step

A transaction that fails to successfully complete its execution will have an **abort** instruction as the last statement

## Concurrent Executions

```
T1: read(A);  
    A := A - 50;  
    write(A);  
    read(B);  
    B := B + 50;  
    write(B).
```

**Transaction T1 transfers \$50 from account A to account B.**

```
T2: read(A);  
    temp := A * 0.1;  
    A := A - temp;  
    write(A);  
    read(B);  
    B := B + temp;  
    write(B).
```

**Transaction T2 transfers 10 percent of the balance from account A to account B**

## Schedule 1

Let  $T_1$  transfer \$50 from  $A$  to  $B$ , and  $T_2$  transfer 10% of the balance from  $A$  to  $B$ . An example of a **serial** schedule in which  $T_1$  is followed by  $T_2$  :

| $T_1$                                                                                                      | $T_2$                                                                                                                               |
|------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------|
| read ( $A$ )<br>$A := A - 50$<br>write ( $A$ )<br>read ( $B$ )<br>$B := B + 50$<br>write ( $B$ )<br>commit | read ( $A$ )<br>$temp := A * 0.1$<br>$A := A - temp$<br>write ( $A$ )<br>read ( $B$ )<br>$B := B + temp$<br>write ( $B$ )<br>commit |

## Schedule 2

A **serial** schedule in which  $T_2$  is followed by  $T_1$  :

| $T_1$                                                                                      | $T_2$                                                                                                               |
|--------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------|
| read (A)<br>$A := A - 50$<br>write (A)<br>read (B)<br>$B := B + 50$<br>write (B)<br>commit | read (A)<br>$temp := A * 0.1$<br>$A := A - temp$<br>write (A)<br>read (B)<br>$B := B + temp$<br>write (B)<br>commit |

- When the database system executes several transactions concurrently, the corresponding schedule no longer needs to be serial.
- If two transactions are running concurrently, the operating system may execute one transaction for a little while, then perform a context switch, execute the second transaction for some time, and then switch back to the first transaction for some time, and so on.
- With multiple transactions, the CPU time is shared among all the transactions



## Schedule 3

- Let  $T_1$  and  $T_2$  be the transactions defined previously. The following schedule is not a serial schedule, but it is **equivalent** to Schedule 1.

| $T_1$                                                                                      | $T_2$                                                                                                               | $T_1$                                                                                          | $T_2$                                                                                                                   |
|--------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------|
| read (A)<br>$A := A - 50$<br>write (A)<br>read (B)<br>$B := B + 50$<br>write (B)<br>commit | read (A)<br>$temp := A * 0.1$<br>$A := A - temp$<br>write (A)<br>read (B)<br>$B := B + temp$<br>write (B)<br>commit | read (A)<br>$A := A - 50$<br>write (A)<br><br>read (B)<br>$B := B + 50$<br>write (B)<br>commit | read (A)<br>$temp := A * 0.1$<br>$A := A - temp$<br>write (A)<br><br>read (B)<br>$B := B + temp$<br>write (B)<br>commit |

Schedule 1

Schedule 3

After this execution takes place, we arrive at the same state as the one in which the transactions are executed serially in the order T1 followed by T2.

The sum  $A + B$  is indeed preserved.

Schedule 4

- The following concurrent schedule does not preserve the sum of “A + B”

| T <sub>1</sub>                                                                         | T <sub>2</sub>                                                                                                | T <sub>1</sub>                                                                                             | T <sub>2</sub>                                                                                                        |
|----------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------|
| read (A)<br>A := A – 50<br>write (A)<br>read (B)<br>B := B + 50<br>write (B)<br>commit | read (A)<br>temp := A * 0.1<br>A := A - temp<br>write (A)<br>read (B)<br>B := B + temp<br>write (B)<br>commit | read (A)<br>A := A – 50<br><br><br><br><br><br>write (A)<br>read (B)<br>B := B + 50<br>write (B)<br>commit | read (A)<br>temp := A * 0.1<br>A := A - temp<br>write (A)<br>read (B)<br><br><br>B := B + temp<br>write (B)<br>commit |

Schedule 1

Schedule 4

After the execution of this schedule, we arrive at a state where the final values of accounts A and B are \$950 and \$2100, respectively.

This final state is an inconsistent state, since we have gained \$50 in the process of the concurrent execution.

Indeed, the sum A + B is not preserved by the execution of the two transactions.